



D.Y. Patil Academic Education Excellence Federation's

Dr. D. Y. Patil Technical Campus

Polytechnic | Engineering | MCA

(Approved by AICTE, Recognized by DTE, Govt. of Maharashtra, Affiliated to Savitribai Phule Pune University, Pune)
Sr.No.32/1/A/7, Near Talegaon Railway Station, A/P Varale, Talegaon Dabhade, Tal-Maval, Dist-Pune 410 507

Website: www.dypatiltes.com Email: principal@dypatiltes.com

AICTE ID: 1- 43358498993 | SPPU PUN CODE: CMMP021930 | ISHE CODE: C -71504 | DTE CODE: 6991

Dr. D. Y. Patil

Founder Dr. D. Y. Patil Group

Dr. Sushant V. Patil

Chairman

Adv. Anuja Sushant Patil

Director

Dr. Suresh Shirbahadurkar

Principal

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

TOPIC: Python Language

Prepared By: Mansi Ghodke

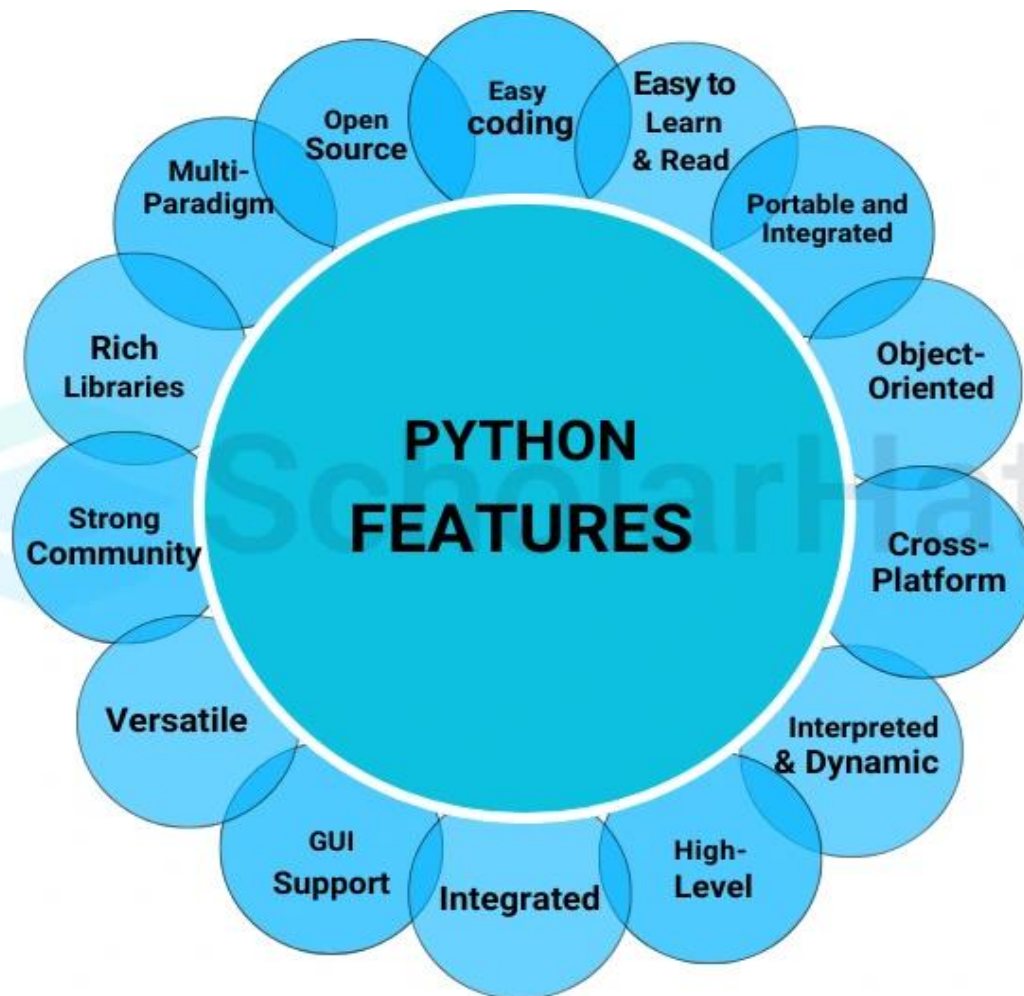
Python Programing Language

What is Python?

Python is a **high-level, general-purpose programming language** known for readable syntax and a large standard library.

Python is a beginner-friendly yet powerful programming language, widely used in data science, AI, web development, and automation. Its core concepts include variables, data types, control flow, functions, data structures, object-oriented programming, exception handling, and file/database management. Mastering these fundamentals gives you the foundation to build anything from small scripts to large-scale applications.

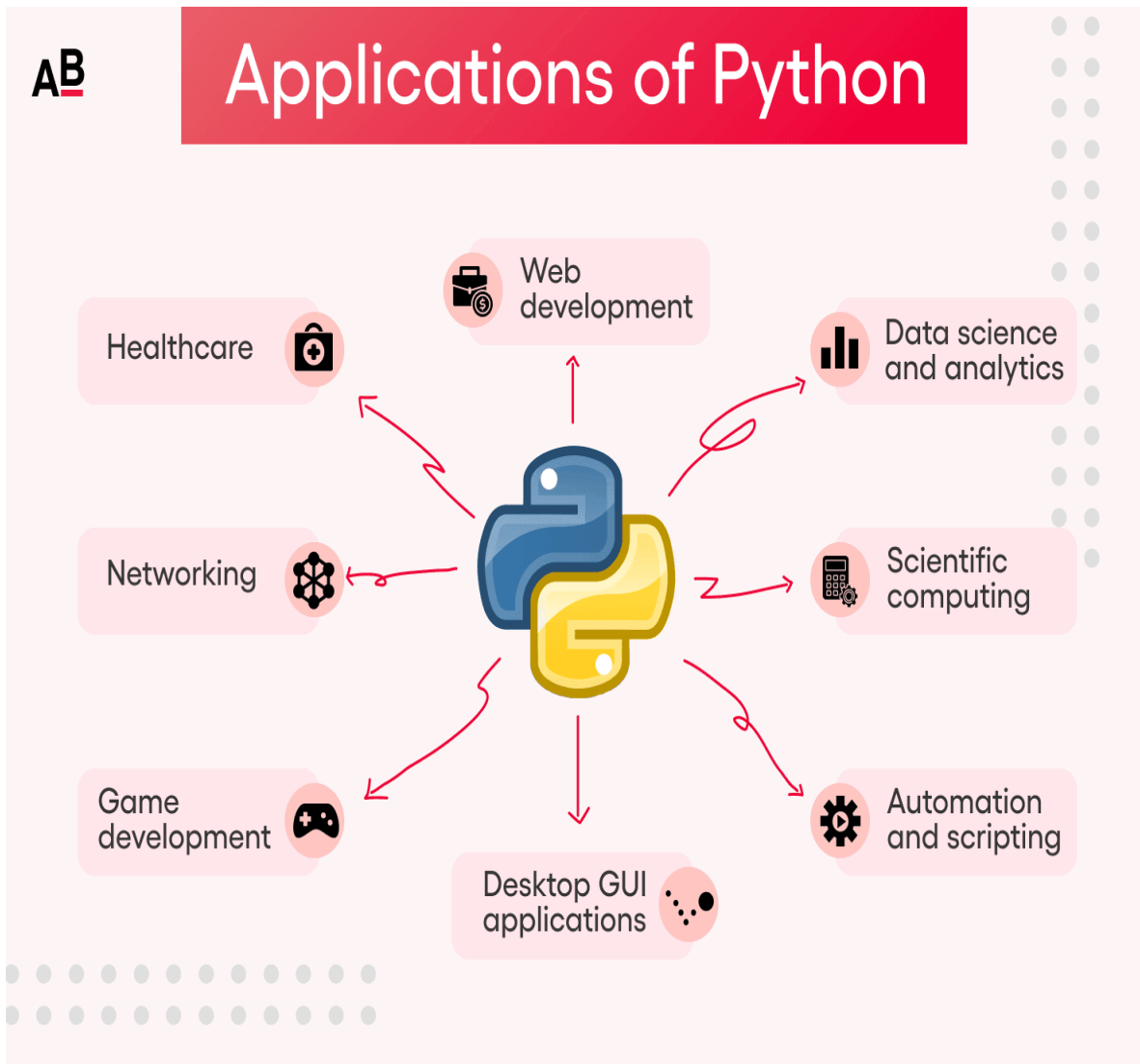
✓ Features Of Python



Key Features of Python

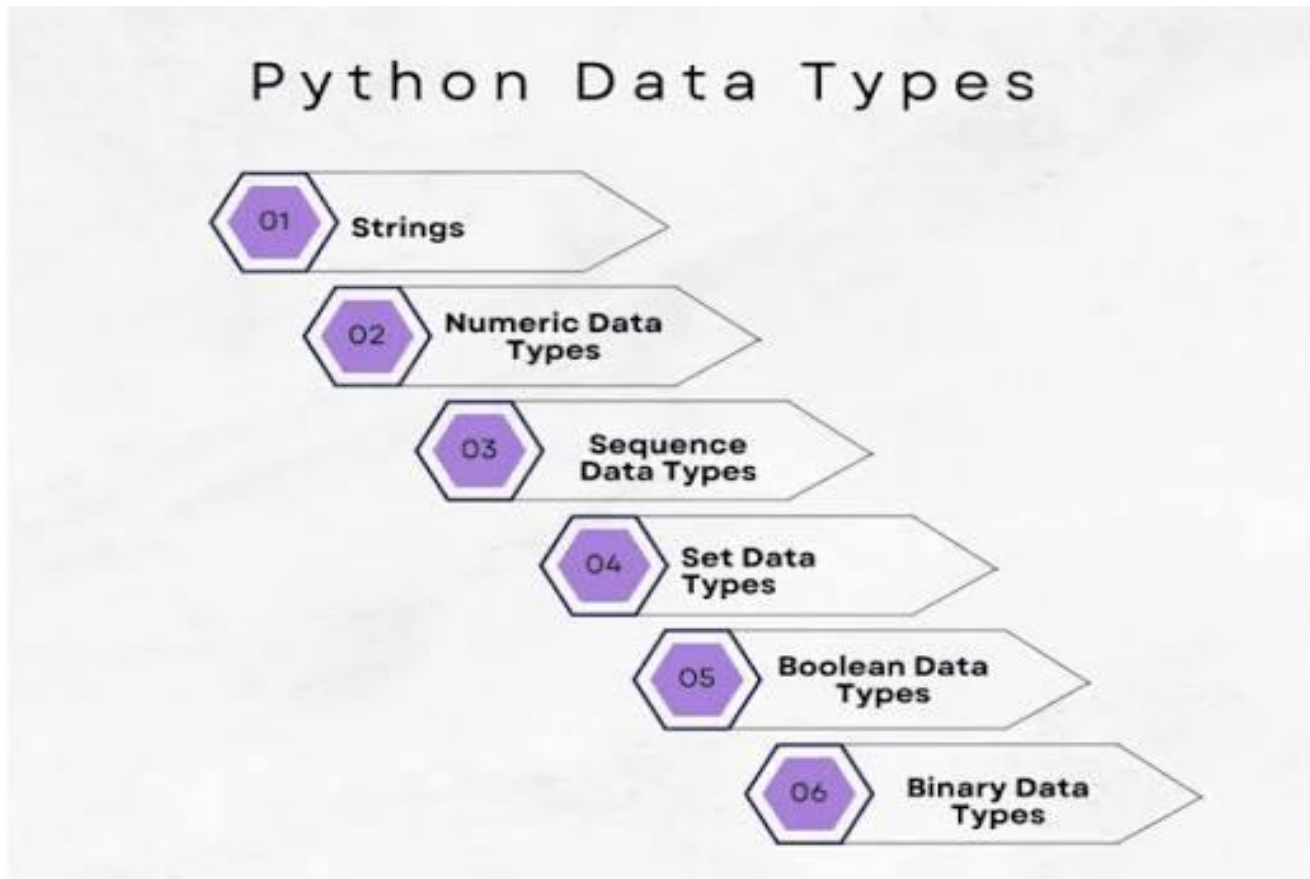
- **Interpreted language:** No compilation step; code runs directly, speeding up development.
- **Object-oriented:** Supports classes, inheritance, and encapsulation.
- **Dynamic typing:** Variables don't need explicit type declarations.
- **Extensive standard library:** Includes modules for file I/O, networking, math, and more.
- **Cross-platform:** Runs on Windows, macOS, Linux, and more.
- **Community support:** Thousands of third-party packages available via PyPI.

✓ Application Of Python



Python is an interpreted, high-level, dynamically typed, and object-oriented programming language renowned for its clean syntax, code readability, and vast library ecosystem. Because it removes compilation steps and enforces indentation instead of curly braces, developers can express complex program design choices with fewer lines of code. python is a high-level, interpreted, and multi-paradigm programming language renowned for its clear readability and clean, intuitive syntax. Because it relies on **indentation instead of curly braces** to mark blocks of code, Python naturally forces developers to write clean, visually organized programs

✓ Python Data Types



✓ Control Flow & Structure

Python programs evaluate conditions and loop through sequences using readable English keywords. [\[1\]](#)

- **Conditional Statements (if-elif-else):** Directs the execution flow based on true/false evaluation.
- **Loops:** for loops iterate over a known collection or range, while while loops run continuously as long as a baseline condition remains true.
- **List Comprehensions:** A clean, single-line alternative for creating new lists out of existing collections.

✓ Functions & Scope

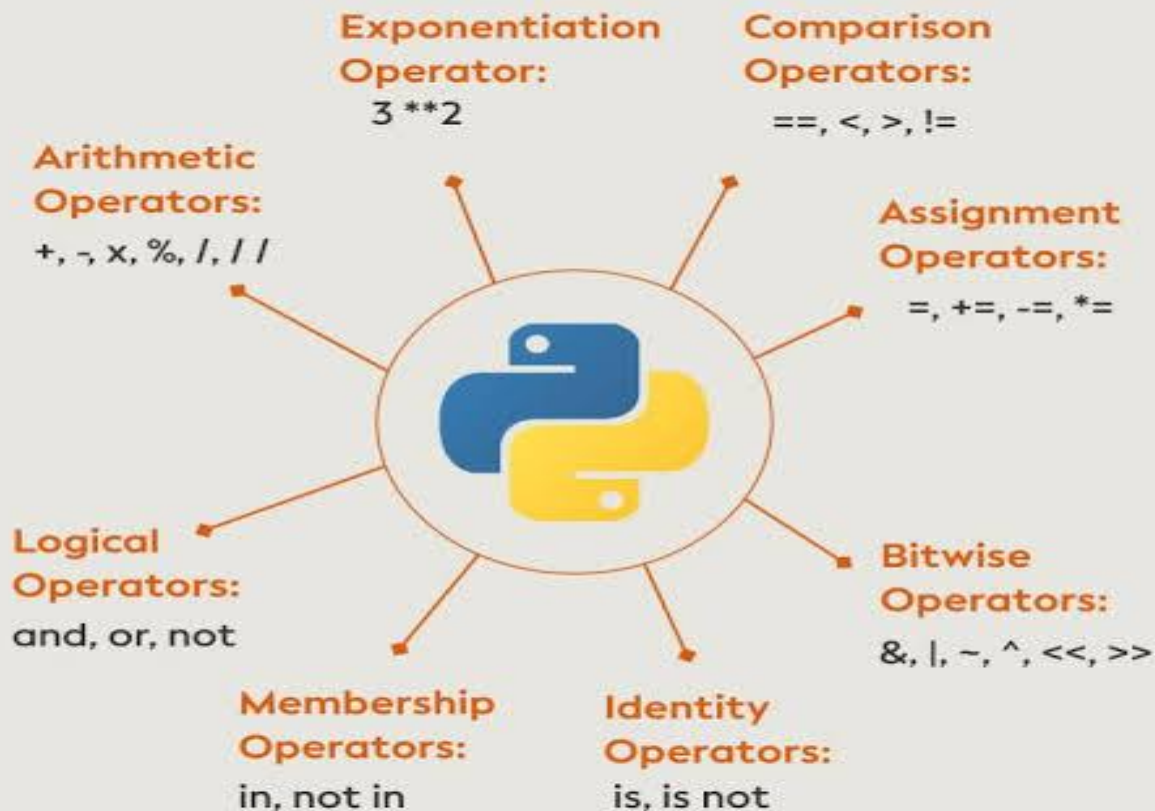
Functions encapsulate reusable blocks of code to prevent repetition.

- **def Keyword:** Declares standard functions.
- **lambda Functions:** Small, anonymous, single-expression functions used for quick computations.
- **Variable Scope:** Variables defined inside a function are *local* to that function. If defined outside, they belong to the *global* scope.

✓ Python Operators



PYTHON OPERATORS



Techie Programmer

©2024

1. Arithmetic Operators

Used to perform standard mathematical calculations. [[1](#), [2](#)]

- **+** **Addition:** $5 + 3 \rightarrow 8$
- **-** **Subtraction:** $5 - 3 \rightarrow 2$
- ***** **Multiplication:** $5 * 3 \rightarrow 15$
- **/** **Division:** Returns a float ($5 / 2 \rightarrow 2.5$)
- **//** **Floor Division:** Rounds down to the nearest integer ($5 // 2 \rightarrow 2$)
- **%** **Modulus:** Returns the remainder ($5 \% 2 \rightarrow 1$)
- ****** **Exponentiation:** Power operator ($5 ** 3 \rightarrow 125$)

2. Assignment Operators

Used to assign or update values to variables. [[1](#), [2](#)]

- **= Assign:** $x = 5$
- **+= Add and Assign:** $x += 3$ (Equivalent to $x = x + 3$)
- **-= Subtract and Assign:** $x -= 3$
- ***= /= / %= Compound Assigns:** Applies the math operation and updates the variable.

3. Comparison (Relational) Operators

Compare two values and always return a Boolean: True or False.

- **== Equal to:** $5 == 3 \rightarrow \text{False}$
- **!= Not equal to:** $5 != 3 \rightarrow \text{True}$
- **> / < Greater / Less than:** $5 > 3 \rightarrow \text{True}$
- **>= / <= Greater / Less than or equal to**

4. Logical Operators

Used to combine conditional statements. [[1](#)]

- **and Logical AND:** Returns True if both statements are true ($x > 2$ and $x < 10$)
- **or Logical OR:** Returns True if at least one statement is true
- **not Logical NOT:** Reverses the result ($\text{not}(\text{True}) \rightarrow \text{False}$)

5. Membership Operators

Used to test if a sequence (like a string, list, or tuple) contains a specific value.

- **in** Returns True if the value is found in the sequence.
- **not in** Returns True if the value is missing from the sequence.

6. Bitwise Operators

Used to perform bit-by-bit operations on binary numbers. [[1](#), [2](#)]

- **& AND | | OR | ^ XOR | ~ NOT | << Left Shift | >> Right**

✓ Concept: Operator Precedence

When multiple operators appear in a single expression (e.g., $x = 5 + 3 * 2$), Python uses **Operator Precedence** to decide what to calculate first.

- **Parentheses ()** have the highest precedence.
- **Math operations** (like $*$ and $/$) happen before addition/subtraction ($+$, $-$).
- **Comparison operators** happen before **Logical operators**. .

Python Concepts Roadmap

Beginner

1. Introduction to Python

- Overview of Python
- Installing Python and an IDE
- Basic syntax and data types

2. Control Flow and Functions

- Conditional statements (if, else, elif)
- Loops (for, while)
- Functions and modular programming

3. Data Structures

- Lists, tuples, and dictionaries
- String manipulation
- Basic input/output

4. File Handling

- Reading and writing files
- Working with different file formats (text, CSV)

5. Exception Handling

- Handling errors and exceptions
- Using try-except blocks

6. Introduction to OOP (Object-Oriented Programming)

- Classes and objects
- Inheritance and polymorphism

7. Introduction to Libraries

- Overview of standard libraries (e.g., math, random)
- Introduction to external libraries (e.g., NumPy for numerical operations)

INTERMEDIATE

1. Advanced OOP

- Encapsulation, abstraction, and inheritance
- Advanced topics in polymorphism

2. Advanced Data Structures

- Sets and frozensets
- Stacks and queues
- Advanced list comprehensions

3. Functional Programming

- Lambda functions
- Map, filter, and reduce functions
- Decorators and closures

3. Functional Programming

- Lambda functions
- Map, filter, and reduce functions
- Decorators and closures

4. Working with APIs

- Making API requests (e.g., using requests library)
- Parsing JSON responses

5. Database Interaction

- Basics of database connectivity (e.g., SQLite)
- CRUD operations with databases

6. Introduction to OOP (Object-Oriented Programming)

- Classes and objects
- Inheritance and polymorphism

7. Introduction to Libraries

- Overview of standard libraries (e.g., math, random)
- Introduction to external libraries (e.g., NumPy for numerical operations)

ADVANCED

1. Advanced Web Development

- RESTful APIs
- Authentication and authorization
- Front-end frameworks (e.g., React, Angular)

2. Concurrency and Parallelism

- Threading and multiprocessing
- Asyncio for asynchronous programming

3. Advanced Libraries and Frameworks

- Deep dive into popular libraries (e.g., Pandas, NumPy, TensorFlow)
- Customizing and extending frameworks

4. Security in Python

- Best practices for secure coding
- Handling common security vulnerabilities

5. Optimization and Performance

- Profiling and optimizing code
- Memory management and optimization techniques

6. Advanced Topics in Data Science

- Machine learning with Python (e.g., scikit-learn)
- Data Visualization with Matplotlib and Seaborn

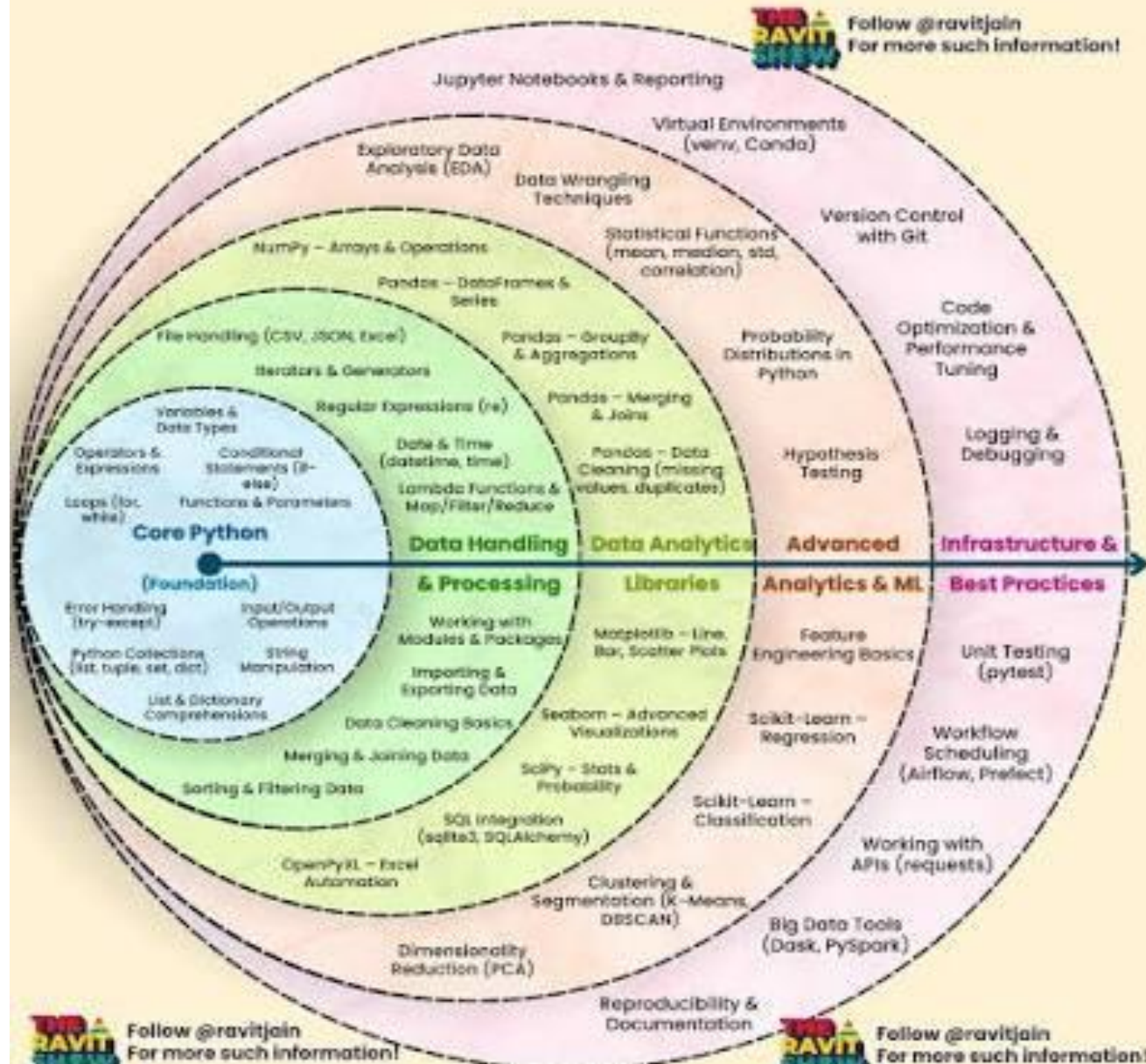
7. Contributing to Open Source Projects

- Understanding open source contribution workflows
- Collaborating on GitHub and other version control system

Python Concepts For Data Analytics



Follow @ravitjain
For more such information!



Follow @ravitjain
For more such information!



Follow @ravitjain
For more such information!

✓ Variables, Memory, and Dynamic Typing

dynamic typing, meaning you do not declare variable types explicitly. The interpreter determines the type at runtime based on the value assigned.

- **Object References:** In Python, variables are labels pointing to objects stored in memory, not the memory slots themselves.
- **Mutability:** Objects are either **mutable** (can be changed in place) or **immutable** (cannot be changed after creation).

```
# Dynamic typing
x = 10      # x is an int
x = "Hello" # x is now a str (perfectly valid)
```

```
# Mutability Demonstration
list_a = [1, 2, 3]
list_b = list_a    # Both point to the same memory object
list_a.append(4)
print(list_b)      # Output: [1, 2, 3, 4] (Lists are mutable)
```

```
num_a = 5
num_b = num_a
num_a += 1
print(num_b)
```

Output: 5 (Integers are immutable; num_a created a new object)

✓ Built-in Data Structures Deep Dive

Python's built-in collections have unique performance characteristics and use cases:

- **Lists ([])**: Ordered, mutable sequences. Excellent for linear collections where items change frequently.
- **Tuples (())**: Ordered, immutable sequences. Faster than lists and protect data from accidental modification. Frequently used to return multiple values from functions.
- **Dictionaries ({k: v})**: Hash tables that store key-value pairs. Lookup, insertion, and deletion times are incredibly fast— $O(1)$ on average.

```
# Dictionary lookup and Set uniqueness
```

```
user_roles = {"alice": "admin", "bob": "editor"}  
print(user_roles["alice"])
```

```
# Output: admin
```

```
unique_ids = {101, 102, 101, 103}  
print(unique_ids)      # Output: {101, 102, 103} (Duplicates removed)
```

✓ Advanced Control Flow & Iterables

Beyond standard if statements, Python uses powerful syntax to process sequences efficiently.

- **List Comprehensions:** A declarative, highly readable way to generate new lists out of existing iterables. They execute faster than standard for loops because they run at C-level speeds internally.
- **Generators:** Functions that use the yield keyword instead of return. They produce items lazily (one at a time), saving massive amounts of RAM when dealing with massive datasets.

```
# List Comprehension: Square even numbers
```

```
numbers = [1, 2, 3, 4, 5, 6]  
squares_of_evens = [x**2 for x in numbers if x % 2 == 0]  
print(squares_of_evens) # Output: [4, 16, 36]
```

```
# Generator Function
```

```
def count_up_to(max_val):  
    count = 1  
    while count <= max_val:  
        yield count  
        count += 1
```

```
counter = count_up_to(1000000) # Evaluates lazily; consumes almost zero memory  
print(next(counter))          # Output: 1  
print(next(counter))          # Output: 2
```

✓ Object-Oriented Programming (OOP)

Python is a multi-paradigm language that fully supports **Object-Oriented Programming** to model real-world scenarios using classes and objects.

- **Classes & Objects:** A Class is a blueprint; an Object is an instance of that blueprint.
- **Encapsulation:** Restricting access to methods and variables by using a single underscore `_` (protected) or double underscore `__` (private/name-mangled).
- **Inheritance:** Creating a child class that inherits attributes and methods from a parent class.
- **Polymorphism:** Allowing different classes to use methods with the identical name but distinct logic.

```
class Animal:
    def __init__(self, name):
        self.name = name # Instance variable

    def speak(self):
        raise NotImplementedError("Subclass must implement this method")

class Dog(Animal):
    def speak(self):
        return f'{self.name} says Woof!'

class Cat(Animal):
    def speak(self):
        return f'{self.name} says Meow!'

# Polymorphism in action
pets = [Dog("Buddy"), Cat("Whiskers")]
for pet in pets:
    print(pet.speak())

# Output:
# Buddy says Woof!
# Whiskers says Meow!
```



Important Concepts in Python Programming



Basics

- Basics syntax
- Variables
- Data types
- Conditionals
- Typecasting
- Exceptions
- Functions
- Lists, Tuples, Sets
- Dictionaries

Advanced

- List
- Comprehensions
- Generators
- Expressions
- Paradigms
- Regex
- Decorators
- Iterators
- Lambdas

OOP

- Classes
- Inheritance
- Methods

Data Science

- NumPy
- Pandas
- Matplotlib
- Seaborn
- Scikit-learn
- TensorFlow
- PyTorch

Data Structures and Algorithms

- Arrays and Linked Lists
- Heaps, Stacks, Queues
- Hash Tables
- Binary Search Trees
- Recursion
- Sorting Algorithms

Web Frameworks

- Django
- Flask
- FastAPI
- Tornado

Package Manager

- PyPI
- pip
- conda

Automation

- File manipulation
- Web scraping
- GUI automation
- Network automation

✓ Practical Applications

Python is widely used in various domains, including:

- **Web Development:** Frameworks like Django, Flask, and Pyramid facilitate the creation of web applications.
- **Data Science and Machine Learning:** Libraries such as NumPy, SciPy, Pandas, TensorFlow, and scikit-learn make Python a popular choice for data analysis and machine learning.
- **Automation and Scripting:** Python's simple syntax and powerful libraries make it ideal for automating repetitive tasks and writing scripts.
- **Scientific Computing:** Tools like SciPy and IPython support scientific research and numerical computations.
- **Software Development:** Python is used for developing software applications, including GUI applications with libraries like Tkinter and PyQt.

Example Code

Here is a simple example of a Python function that prints the Fibonacci series up to a given number `n`:

```
def fib(n):  
    a, b = 0, 1  
    while a < n:  
        print(a, end=' ')  
        a, b = b, a + b  
    print()
```

```
# Call the function with n = 1000  
fib(1000)
```

Function & Syntax

- **Dynamically Typed:** Variables do not need explicit type declarations; Python determines the type automatically at runtime.
- **Indentation Rules:** Python uses clean whitespace instead of curly brackets ({}) or semicolons to define code blocks.
- **Primitive Types:** Fundamental built-in data types including Integers (int), Floats (float), Strings (str), and Booleans (bool).
- **Type Casting:** Manually converting one data type to another using functions like `int()`, `str()`, or `float()`.

2. Control Flow & Logic

- **Conditionals:** Branching logic using the `if`, `elif`, and `else` statements.
- **Definite Loops (for):** Iterating over a specific, finite sequence (like a list or a mathematical range).
- **Indefinite Loops (while):** Continuous execution of a block of code as long as a specified condition remains true.
- **Loop Modifiers:** Keywords used to alter loop behavior: `break` to exit completely, `continue` to skip to the next iteration, and `pass` as a syntactic placeholder.

3. Core Data Structures

- **Lists:** Ordered, mutable (changeable) sequences that allow duplicate items: `['apple', 'banana']`.
- **Tuples:** Ordered, immutable (unchangeable) sequences used for fixed data collections: `(10, 20)`.
- **Dictionaries:** Highly optimized collections of key-value pairs mapping unique keys to data: `{"id": 101}`.
- **Sets:** Unordered collections containing entirely unique elements, ideal for mathematical operations like unions or intersections: `{1, 2, 3}`.

4. Functions & Functional Programming

- **User-Defined Functions:** Reusable blocks of code isolated by the `def` keyword.
- **Scope Resolution:** How Python looks up variables based on the **LEGB rule** (Local, Enclosing, Global, Built-in).
- **Arguments & Parameters:** Flexible data inputs including positional arguments, keyword arguments, default values, and variable-length inputs (`*args`, `**kwargs`).
- **Lambda Functions:** Small, anonymous, single-expression functions: `lambda x: x * 2`.
- **First-Class Functions:** The treatment of functions as standard objects, meaning they can be passed as arguments, returned from other functions, and assigned to variables.
- **Built-in Higher-Order Functions:** Utilities like `map()`, `filter()`, and `reduce()` that process iterables functionally.

5. Advanced Data Manipulation

- **Comprehensions:** Concise, inline syntaxes used to build new **lists, dictionaries, or sets** elegantly out of existing iterables.
- **Slicing:** A powerful indexing syntax (`sequence[start:stop:step]`) used to extract specific sub-sections of strings, lists, or tuples.
- **Unpacking:** Extracting items from sequences directly into distinct variables in a single line: `x, y = (1, 2)`.

6. Object-Oriented Programming (OOP)

- **Classes & Objects:** A **Class** acts as a blueprint, while an **Object** is a concrete instance generated from that blueprint.
- **The self Keyword:** A reference representing the specific instance of the object currently being modified or accessed.
- **Dunder/Magic Methods:** Special underlying methods wrapped in double underscores (like `__init__` for initialization or `__str__` for string representation) that overload standard Python operators.
- **Four Pillars of OOP:**
 - **Inheritance:** Enabling a child class to adopt the attributes and methods of a parent class.

- **Polymorphism:** Allowing different classes to share the same method name but execute unique underlying behaviors.
- **Encapsulation:** Restricting direct access to an object's data to protect its internal state (using private variables prefixing with `_` or `__`).
- **Abstraction:** Hiding complex implementation details and showing only essential features using abstract base classes (abc module).

7. File I/O & Resource Management

- **File Operations:** Reading from and writing to external files using methods like `.read()` and `.write()`.
- **Context Managers (with statement):** A reliable architectural pattern that guarantees resources (like file streams or database connections) are automatically closed or cleaned up, even if errors occur.

8. Robustness & Error Handling

- **Exception Handling:** Using `try`, `except`, `else`, and `finally` blocks to intercept runtime errors gracefully without terminating the program.
- **Custom Exceptions:** Creating personalized error classes by inheriting from Python's built-in `Exception` class.
- **Assertions:** Using the `assert` statement to test internal programming assumptions during development.

9. Python Internals & Execution Model

- **Memory Management:** Automated allocation using a built-in **Garbage Collector** and **Reference Counting** to free up unused memory.
- **GIL (Global Interpreter Lock):** A fundamental mutex mechanism that restricts execution to exactly one thread at a time per process, ensuring thread-safe memory management.
- **Mutable vs. Immutable Internals:** Understanding how items like integers and strings point to rigid, unchanging addresses in memory, whereas lists can alter their internal structures dynamically.

10. Advanced Structural & Behavioral Metaprogramming

- **Generators:** Custom iterators that yield values one at a time using the `yield` keyword, saving memory by streaming data rather than loading whole lists into RAM.
- **Decorators:** Functions that wrap around other functions or classes to modify or extend their behavior without altering the actual source code.
- **Iterators & Iterables:** An **Iterable** is an object capable of returning its elements one by one; an **Iterator** is the underlying engine that tracks the current state using the `__next__()` method.
- **Metaclasses:** Deep-level classes that act as blueprints for standard classes, controlling how classes themselves are constructed.

11. Concurrency, Distribution, & Structuring

- **Modules & Packages:** Splitting code across distinct `.py` files (**Modules**) or structured directories containing multiple modules (**Packages**).
- **Virtual Environments:** Tools (`venv`, `conda`) used to isolate third-party library dependencies cleanly between projects.
- **Asynchronous Programming (asyncio):** Writing concurrent, single-threaded code using `async` and `await` keywords to prevent network or I/O blocks.

- **Multithreading vs. Multiprocessing:** Choosing between threading for performance boosts in I/O-bound tasks versus multiprocessing to bypass the GIL and utilize multiple CPU cores for heavy computation.

✓ Loops

1. The for Loop

A for loop is used when you know in advance how many times you want to run the code, or when you want to step through items in a collection (like a list, tuple, or string).

A. Iterating Over a Range of Numbers

The range() function generates a sequence of numbers.

Python

```
# Counts from 0 to 4 (5 is excluded)
for i in range(5):
    print(f"Count: {i}")
```

B. Iterating Over a List

python

```
fruits = ["apple", "banana", "cherry"]
```

```
for fruit in fruits:
    print(f"I like {fruit}")
```

2. The while Loop

A while loop runs as long as a specific condition evaluates to True. It is ideal when you don't know exactly how many iterations it will take to finish.

python

```
countdown = 3
```

```
while countdown > 0:
```

```
    print(countdown)
```

```
    countdown -= 1 # Crucial: updates the condition so the loop eventually stops
```

```
print("Blast off! 🚀")
```

3. Loop Control Statements

You can alter the behavior of a loop mid-execution using three key control statements:

A. **break** (Exit the loop completely)

Stops the loop entirely and jumps to the code outside it.

Python

```
for num in range(1, 10):  
    if num == 5:  
        break # Stops the loop completely when it hits 5  
    print(num)  
# Output: 1, 2, 3, 4
```

B. **continue** (Skip the current round)

Skips the rest of the current iteration and jumps directly to the next loop cycle.

python

```
for num in range(1, 6):  
    if num == 3:  
        continue # Skips printing 3  
    print(num)  
# Output: 1, 2, 4, 5
```

B. **pass** (The placeholder)

Does absolutely nothing. It is used as a syntactic placeholder when code is syntactically required but you haven't written the logic yet.

python

```
for num in range(5):  
    pass # Keeps Python from throwing an error until you write code here
```

4. Advanced Loop Tricks

The **enumerate()** Function

If you need both the **index** (position) and the **value** of items in a list, wrap the list in `enumerate()`.

python

```
languages = ["Python", "Java", "C++"]
```

```
for index, lang in enumerate(languages):  
    print(  
f"Rank {index + 1}: {lang}")
```

5.The else Clause in Loops

Python loops have an optional else block. The else code runs **only if the loop completes naturally** without encountering a break statement.

```
python
for item in:
    print(item)
else:
    print("Loop finished smoothly!")
```

1. Python's Memory Model: Mutable vs. Immutable

In Python, **everything is an object**. When you create a variable, you are not creating a slot in memory that holds a value; instead, you are creating a **reference (pointer)** to an object in memory.

- **Immutable Objects:** Cannot be changed after creation. If you try to modify them, Python creates a brand-new object at a different memory address.

- *Types:* int, float, str, tuple, bool.

- **Mutable Objects:** Can be changed in place without changing their memory location.

Code Demonstration:

```
python
# --- Immutable Example ---
x = 10
print(id(x)) # Prints the unique memory address of 10

x = x + 1
print(id(x)) # Prints a NEW address. The number 10 was not changed; x now points to 11.

# --- Mutable Example ---
my_list = [1, 2, 3]
print(id(my_list)) # Memory address of the list

my_list.append(4)
print(id(my_list)) # SAME address! The existing list was modified directly.
Use code with caution.
```

2. Variable Scope and the LEGB Rule

When you reference a variable name, Python looks for it in a strict visual hierarchy known as the **LEGB rule**. It stops searching at the first match it finds:

1. **L (Local):** Names assigned inside a function (def or lambda) and not declared global.

2. **E (Enclosing):** Names in the local scope of any enclosing functions (relevant in nested functions/closures).
3. **G (Global):** Names assigned at the top level of a module file, or declared global inside a function.
4. **B (Built-in):** Predefined names in Python (like print, len, range, ValueError).

Code Demonstration:

```
python
x = "Global X" # Global Scope

def outer_function():
    x = "Enclosing X" # Enclosing Scope

    def inner_function():
        x = "Local X" # Local Scope
        print(x) # Prints: Local X

    inner_function()
```

```
outer_function()
Use code with caution.
```

Note: If you comment out `x = "Local X"`, the program will look outward and print "Enclosing X".

3. Iterators and Generators (Memory Management)

When handling large data streams, loading millions of records into a regular list will crash your computer's RAM. Python solves this with **Generators**.

- **Iterable:** An object that can return an Iterator (e.g., a list or a string).
- **Iterator:** The engine that remembers its state and fetches the next item using the `__next__()` method.
- **Generator:** A special function that yields values one at a time using the `yield` keyword instead of `return`. It pauses execution until the next value is requested, using virtually zero memory.

Code Demonstration:

```
python
# Traditional List Approach (Uses lots of RAM if range is huge)
def standard_squares(n):
    result = []
    for i in range(n):
        result.append(i ** 2)
    return result

# Generator Approach (Uses nearly zero RAM)
def generator_squares(n):
    for i in range(n):
        yield i ** 2 # Pauses here and returns value on demand

# Using the generator
gen = generator_squares(1000000)
print(next(gen)) # Output: 0
```

```
print(next(gen)) # Output: 1
# The remaining 999,998 numbers haven't even been calculated yet!
Use code with caution.
```

4. Closures and Decorators

A **Decorator** is a powerful design pattern that allows you to modify or extend the behavior of a function or class without permanently changing its source code. Decorators are built on top of **Closures** (nested functions that remember variables from their outer scope even after the outer function has finished executing).

Code Demonstration:

Let's build a decorator that measures how long a function takes to execute:

```
python
import time

# The Decorator function
def execution_timer(func):
    def wrapper(*args, **kwargs):
        start_time = time.time()    # 1. Take start time
        result = func(*args, **kwargs) # 2. Run the actual function
        end_time = time.time()      # 3. Take end time
        print(f'Function '{func.__name__}' took {end_time - start_time:.4f} seconds to run.')
        return result               # 4. Return the function's output
    return wrapper

# Applying the decorator using the '@' syntax
@execution_timer
def heavy_calculation():
    total = 0
    for i in range(10_000_000):
        total += i
    return total

# Run the function
heavy_calculation()
Use code with caution.
```

5. Advanced Object-Oriented Programming (OOP)

Python supports robust OOP principles, including complex concepts like **Dunder Methods** (Magic Methods) and **Multiple Inheritance**.

- **Dunder Methods (__double_underscore__):** These allow you to hook into Python's built-in operators. For instance, defining `__add__` lets you use the `+` sign on your own custom objects.

Code Demonstration:

```
python
class Vector:
```

```

def __init__(self, x, y):
    self.x = x
    self.y = y

# Overloading the "+" operator
def __add__(self, other):
    return Vector(self.x + other.x, self.y + other.y)

# Customizing how the object prints out text
def __str__(self):
    return f"Vector({self.x}, {self.y})"

v1 = Vector(2, 4)
v2 = Vector(5, 3)

# Because we used __add__, Python lets us do this:
v3 = v1 + v2
print(v3) # Output: Vector(7, 7)
Use code with caution.

```

6. The Context Manager (with statement)

When dealing with external resources like system files, network connections, or databases, you must always release them when done. If your code crashes before closing them, it causes memory leaks.

A **Context Manager** guarantees resource cleanup using the with statement, executing setup code (`__enter__`) and teardown code (`__exit__`) behind the scenes automatically.

Code Demonstration:

```

python
# The Pythonic, safe way to write to a file
with open("log.txt", "w") as file:
    file.write("Logging critical system application details.")
# The file is automatically CLOSED right here, even if the code inside the block

```

